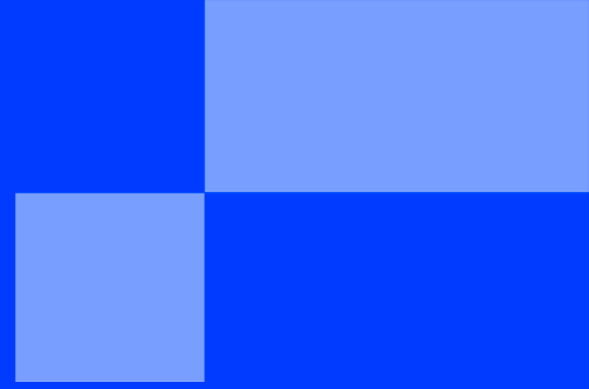
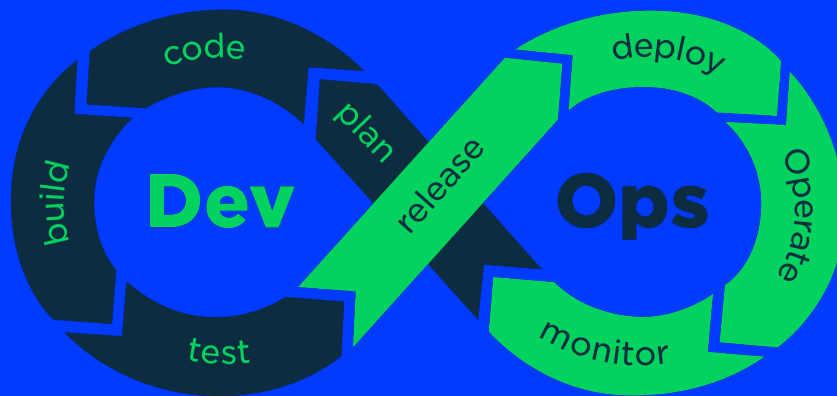




OCTOPUS - BOOTSTRAP

< DIVE INTO THE CONFIGURATION
MANAGEMENT SEA AND MIND-CONTROL
MACHINES />



OCTOPUS - BOOTSTRAP

Let us get straight to the point: configuring machines is tedious to do manually.

And when it comes to configuring 5, 10, or even 100 or 1,000 machines, "tedious" becomes an euphemism.

Thus comes back the notion of automation to save the day! But not in the same way as with the *my_marvin* project.

Here, we are talking about a way to facilitate the configuration of machines, and a way to manage all of that: *configuration management*.

During this bootstrap, you are going to become familiar with one of the leading configuration management tool: *Ansible*, and its associated tool *Ansible Playbook*.

Step 0 – Ansible aims at the incredible!

A little quiz to start with

Traditions are traditions, one may not skip them! Here is a little quiz that you need to answer with your friends before going on an adventure:

- ✓ What is Ansible?
- ✓ What is an Ansible *playbook*?
- ✓ Why using Ansible instead of just bare shell scripts?
- ✓ In what way is Ansible different than Jenkins?
- ✓ Can octopuses help you in your DevOps journey? (octocats do not count)

Alright, have you answered all the questions above? For real? Then, onto the practical stuff!

Setup time

During this bootstrap, as well as with the Octopus project itself, you will need multiple virtual machines.

You can run them locally, or use a cloud provider (the latter is recommended and easier).

- ✓ Install Ansible and Ansible Playbook on your own machine.
- ✓ Start 3 virtual machines running on Debian 13, and upload your public SSH key to them (so you can connect without entering a password).



If you are running local virtual machines, remember to **install openssh-server** on them (so you can connect to them by SSH).



You can use the `ssh-copy-id` command to upload your public SSH key to the remote machines.

Networking

If you are running the virtual machines on a cloud provider (as you should), you can skip this section.

If you are running local virtual machines, make sure that the virtual machines **can communicate with each other** in regards to networking.

You need two network interfaces on each virtual machine:

- ✓ one for the **public** network (for the internet access);
- ✓ one for the **private** network (for the communication between the virtual machines).

In order to reliably connect to the virtual machines from your own machine (and from the virtual machines themselves), you need to set up a **static IP address** for each virtual machine.

You can use the `ping` command to check if the virtual machines can communicate with each other.

```
Terminal
$> ping <ip-address>
```

Step 1 – Taking inventory

Before describing your infrastructure, you will need to list targeted servers into an *inventory file*.

We often create an inventory for each environment: development, staging, production...
For easier manipulation, it is recommended to organize servers in groups.

✓ Create your first inventory:

- Group `web`, containing 2 hosts respectively named `web-1` and `web-2`.
- Group `redis`, containing 1 host named `cache-1`.

You can now check if hosts are up and responding:

```
Terminal
$> ansible -i my-inventory web -m ping
$> ansible -i my-inventory redis -m ping
$> ansible -i my-inventory all -m ping
```

1. Here, `ping` is an Ansible module. Can you find a way to execute the `uptime` Linux command on all 3 hosts at the same time?
2. Can you reboot these machines simultaneously?
3. Using the `apt` Ansible module, can you install `htop` everywhere, then `nginx` only on web servers?
4. We do not need `htop` anymore, can you uninstall it using the `apt` module?
5. Can you ensure that Nginx will be automatically started at system start, using the `service` module?

Step 2 – Role-playing development

Ansible will help you configure a lot of machines and ensure that the expected state is reached. But executing tons of Ansible commands is not very funny! Starting from now, you will set in stone your configuration in YAML files (yes, YAML again!).

In the Ansible architecture, a *playbook* calls multiple *roles*, that call multiple *tasks*. Playbooks must be as simple as possible, and call reusable roles.

Roles must do one thing and do it well (e.g: installing and configuring a database).

If you need multiple databases on different machines, this Ansible role will use *variables* to configure different usernames and passwords.

1. Create your first playbook, and a role `base`, which must:
 - ✓ be executed on every host;
 - ✓ update the OS if necessary;
 - ✓ install some essential packages, such as: `apt-transport-https`, `ca-certificates`, `git`, `curl`, `unzip`, `zsh`, ...
 - ✓ set the timezone (e.g.: `Europe/Paris` if your server is based in France);
 - ✓ set `zsh` as the root shell.
2. Create a `redis` role that installs... well... Redis (on the cache server). ;)
3. Create a `nodejs` role, responsible for installing Node.js and NPM on both web servers.
4. Create a `deploy` role that:
 - ✓ uploads the application provided in this bootstrap to both web servers;
 - ✓ unzips application;
 - ✓ installs dependencies.
5. Right now, the Node.js application is still not running. Write your own SystemD service and upload it to `/etc/systemd/system/my-awesome-app.service`.



None of these tasks need the `command`, `shell`, or `raw` modules!

Step 3 – Variables, for even more laziness

In the last step, your Node.js app has started. But as you probably figured out, the application configuration file has been hardcoded.

1. Can you create a "template" of `config.json`, using Ansible and Jinja2, then upload it on the server from the `deploy` role?
2. Configure the HTTP port and Redis hostname in `group_vars/all.yml`. This is the right place to write your (global) variables when using Ansible.
3. Now, try to set the Redis hostname **dynamically**, based on the inventory.
4. The SystemD service must be restarted **only** when the application is updated. If you deploy the app many times without changing anything, the service should not restart. Try to find a way to *handle* (wink wink) that.

Step 4 – Keep secrets... secret

Ansible comes with a tool that encrypts passwords: *Vault*.

- ✓ Can you add a `password` variable containing the **encrypted string** `octopus-secrets` to the `group_vars/all.yml` file?
- ✓ Try now to print it into `/tmp/octoleak.txt` on the `redis` machine.

What a long and dense bootstrap!

You should have gained more or less control over your (Ansible) tentacles now.

You are ready to dive into the An-sea-ble, and automate, from a single computer, an entire fleet of machines for you to control!

Good luck with that! :)

Step -1 – A secret world

Nothing to do here, but here is a little tip to help you. ;)

The following task prints the value of the `name` variable.

```
- name: Create variable
  set_fact:
    name: world

- name: Print debug
  debug:
    msg: "Hello {{ name }}"
```

v 1.3

{EPITECH}